Dynamic Pricing
Intelligence Platform

Image-to-Price: Vision AI + Real-Time Web Scraping + GenAI Pricing Engine

1. PROJECT OVERVIEW

1.1 Background & Motivation

Pricing a product competitively in today's e-commerce landscape is a continuous, data-intensive challenge. A seller who prices too high loses customers to competitors; price it too low and they sacrifice margin. Dynamic pricing — the practice of continuously adjusting prices based on real-time market intelligence — is now a core capability for any serious online business.

This project challenges you to build a fully automated, AI-powered Dynamic Pricing Intelligence Platform. The system takes a single product image as input, uses computer vision to identify the product, autonomously scrapes live pricing data across multiple e-commerce websites, analyses market conditions using ML models, and uses a Generative AI engine to recommend the optimal selling price — streamed live to the user interface.

No pre-loaded catalogs. No hardcoded product lists. The system must determine what the product is and find its price on the internet — entirely from an image alone.

1.2 Problem Statement

"A seller uploads a photo of any product. Within 60 seconds, the system identifies the product using Vision AI, scrapes live prices from 5+ e-commerce sites, runs ML-based market analysis, and streams a GenAI pricing recommendation — all triggered by a single image upload."

1.3 High-Level Goals

- **Accept a product image as the ONLY input — no text product names, no SKUs, no manual entries.**
- **Use a Vision AI / Multimodal LLM to identify the product (name, brand, category, specs) purely from the image.**
- **Dynamically construct scraping targets based on the identified product — no hardcoded URLs or site lists.**
- **Scrape live prices from at least 5 major e-commerce platforms using real-time parallel web scraping.**
- **Apply ML models to analyse price distribution, detect outliers, cluster price tiers, and estimate demand signals.**
- **Use a GenAI engine to produce a streamed pricing recommendation with detailed, structured rationale.**

- **Deliver a real-time streaming UI where price cards appear as each scraper completes — not all at once at the end.**
- **Expose full observability: scraper health, vision latency, model confidence, price freshness.**

2. SCOPE & CONSTRAINTS

2.1 In Scope

- Image-based product identification using a Vision-capable LLM (GPT-4o, Claude 3.5 Sonnet, or LLaVA via Ollama).
- Real-time web scraping of e-commerce sites on-demand when a user uploads an image.
- ML-based price analysis: outlier detection, K-Means price clustering, demand signal estimation.
- GenAI-powered pricing recommendation streamed token-by-token to the UI via SSE.
- Streaming UI: price cards appear one-by-one as scrapers complete; live price distribution chart updates incrementally.
- Price history persistence: every scrape is stored so trends can be displayed for repeat queries.
- Full observability: Prometheus metrics, Grafana dashboards, and structured JSON logs with trace IDs.
- Containerised deployment via Docker Compose — single startup command for the entire system.

2.2 Out of Scope

- Automated purchasing, cart management, or checkout on external sites.
- Drug-food or drug-disease interactions (DDI only from Project 1).
- Currency conversion (scope to a single region — USD or INR).
- Mobile app development.
- Integration with a seller's existing ERP or product catalog system.

2.3 Hard Constraints

Image-Only Input	The system MUST identify the product from the image alone. Accepting a text product name as a shortcut bypasses the core challenge and is not acceptable.

Live Scraping Only	All prices must be scraped live at query time. Pre-cached static price databases cannot be used as the primary data source.

Streaming Required	Scraping results must stream to the UI as they arrive — do not hold all results and display them together at the end.

No Scraping APIs	You may NOT use paid scraping APIs (ScraperAPI, Bright Data, Apify, etc.). Write your own scrapers using Playwright, httpx, and BeautifulSoup.

Solo Work	All code, architecture decisions, and documentation must be entirely your own.

3. SYSTEM ARCHITECTURE

3.1 Architecture Overview

The platform operates as a six-stage sequential pipeline with streaming data flowing from image upload to the GenAI recommendation. Every stage emits events — either to a task queue or directly via SSE to the browser. The key design principle is that the UI never waits for all stages to complete; it renders incrementally as data arrives.

```
┌──────────────────────────────────────────────────────────────────┐
│                   USER UPLOADS PRODUCT IMAGE                     │
└──────────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────────┐
│  STAGE 1  —  Vision AI  Product Identification                   │
│  Input : Raw image bytes (JPG / PNG / WebP)                      │
│  Output: { product_name, brand, category, specs, search_terms,   │
│            confidence }                                          |
└──────────────────────────────────────────────────────────────────┘
                            │
                            ↓

┌──────────────────────────────────────────────────────────────────┐
│  STAGE 2  —  Dynamic Scraping Orchestrator                       │
│  • Spawns N async scrapers in parallel (one per e-com site)      │
│  • Each scraper runs independently with timeout + retry logic    │
│  • As each scraper succeeds → immediately pushes SSE event       │
└──────────────────────────────────────────────────────────────────┘
                            │
                            ↓  (streaming price events to UI)
┌──────────────────────────────────────────────────────────────────┐
│  STAGE 3  —  Price Normalisation & Persistence                   │
│  • Strips currency symbols, handles price ranges                 │
│  • Flags anomalous prices (Isolation Forest)                     │
│  • Stores normalised records in TimescaleDB (price history)      │
└──────────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────────┐
│  STAGE 4  —  ML Price Analysis Engine                            │
│  • K-Means clustering → Budget / Mid-range / Premium tiers       │
│  • Demand signal estimation (reviews, ratings, badges)           │
│  • Outputs: { market_min, market_avg, market_max,                │
│               recommended_range, competitive_score }             │
└──────────────────────────────────────────────────────────────────┘
                            │
                            ↓
┌──────────────────────────────────────────────────────────────────┐
```

```
│  STAGE 5  —  GenAI Pricing Recommendation Engine                │
│  • Receives: ML output + all scraped prices + product identity │
│  • LLM generates structured pricing rationale                  │
│  • Response streams token-by-token via SSE to the browser      │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌──────────────────────────────────────────────────────────────────┐
│  STAGE 6  —  React Streaming Dashboard                          │
│  • Price cards appear one-by-one as scrapers complete           │
│  • Bar chart updates live with each new data point              │
│  • GenAI recommendation text streams token-by-token             │
│  • Price history trend chart from TimescaleDB                   │
└──────────────────────────────────────────────────────────────────┘
                              ↑   Metrics exported at every stage
          [ Prometheus + Grafana + Loki  Observability Stack ]
```

3.2 Component Deep Dives

3.2.1 Vision AI — Product Identification Service

This is the most critical component. If the product is misidentified, every downstream stage produces wrong results. The service must:

- Accept the uploaded image via multipart form upload and validate it (format, size < 10MB, not corrupted).
- Send the image to a Vision-capable LLM. Supported options: GPT-4o, Claude 3.5 Sonnet, or LLaVA served locally via Ollama.
- Use a carefully engineered prompt that instructs the model to return a strict JSON schema (see below).
- Cache results by image MD5 hash — re-uploading the same image must not trigger another expensive LLM call.
- Return a confidence score: if confidence is below 0.5, the system must surface a warning to the user and ask for manual correction.

Required JSON output schema from the Vision LLM:

```
{
  "product_name"  : "Apple AirPods Pro (2nd Generation)",
  "brand"         : "Apple",
  "category"      : "Electronics > Audio > Wireless Earbuds",
  "key_specs"     : ["Active Noise Cancellation", "USB-C", "MagSafe Case"],
  "search_queries": [
      "Apple AirPods Pro 2nd gen",
      "AirPods Pro USB-C MagSafe",
      "Apple MTJV3LL/A"
  ],
  "confidence"    : 0.94,
  "notes"         : "Identified from Apple logo, distinctive oval case shape, and USB-C port"
}
```

3.2.2 Dynamic Scraping Orchestrator & Individual Scrapers

Once the product is identified, the orchestrator launches all scrapers in parallel as async coroutines. You must implement scrapers for at least 5 platforms:

Platform	Scraping Strategy	Anti-bot Challenge
Amazon	httpx + CSS selectors on search results page	Rate limiting, user-agent detection — rotate UA pool
Flipkart	Playwright (headless) — prices rendered via JS	Heavy JS rendering — Playwright mandatory
eBay	eBay Browse API (free) + HTML scraping fallback	Relatively open — API recommended over raw scrape
Walmart	Playwright + network request intercept for price JSON	Bot detection — use playwright-stealth plugin
Best Buy	httpx + reverse-engineered internal JSON endpoint	API key embedded in page source — parse dynamically
Croma / Tata	httpx + BeautifulSoup — standard HTML	Minimal anti-bot — straightforward to scrape

Anti-bot techniques you MUST implement across all scrapers:

- Rotating user-agent pool: maintain 20+ realistic browser UA strings; randomise per request.
- Request jitter: random delay of 1–4 seconds before each scraper starts (do not launch all simultaneously).
- Playwright stealth mode: use playwright-stealth or undetected-playwright for JS-heavy sites.
- Exponential backoff with retry: max 3 retries per scraper; double the wait time between each retry.
- Per-scraper circuit breaker: if a site fails 3 consecutive times across all sessions, mark it unavailable for 10 minutes and stop retrying.
- Per-scraper timeout budget: each scraper has a hard 25-second timeout. After that it is marked as FAILED regardless.
- Product validation: verify that the scraped product title actually matches the identified product (fuzzy string match >= 70% similarity) before accepting the price.

3.2.3 Price Normalisation Service

Raw scraped prices arrive in inconsistent formats. This service must:

- Strip currency symbols ($ / Rs / £ / €) and convert to base currency.
- Handle price ranges (e.g., '$29.99 – $34.99') — extract min and max, use min as the listed price.
- Flag clearly anomalous prices using Isolation Forest — e.g., an iPhone priced at $5 is a scraping artefact, not a market price.
- Extract enrichment data where available: seller/vendor name, star rating, review count, estimated shipping cost, 'in stock' status.
- Write normalised records to TimescaleDB with full timestamps for historical trend tracking.

3.2.4 ML Price Analysis Engine

- After prices arrive from enough scrapers (configurable threshold: minimum 3 sources), the ML engine performs market analysis:
- **K-Means Clustering (k=3): Segment collected prices into Budget / Mid-range / Premium tiers. Report which cluster each scraped price falls into.**
- **Outlier Detection (Isolation Forest): Flag prices that are statistical anomalies. Exclude flagged prices from the recommendation calculation.**
- **Demand Signal Estimation: Use review counts, ratings, and 'bestseller' badges as proxy demand signals. Compute a composite demand score (0–100).**
- **Competitive Positioning Score: Composite metric (0–100) indicating how competitive the recommended price is vs. the current market.**
- **Output: A price recommendation object { low, target, high, confidence_interval, strategy }.**

3.2.5 GenAI Pricing Recommendation Engine (Streaming)

- After ML analysis, the GenAI engine generates a human-readable pricing recommendation streamed token-by-token. It receives: identified product details, all scraped prices with source and timestamp, and the full ML analysis output.
-
- The LLM must produce a structured streaming response covering these five sections:
- Market Summary: Current price landscape — what does the competition look like?
- Recommended Selling Price: A single target price with concise justification.
- Pricing Strategy: Penetration (go low to capture market share) / Competitive (match market) / Premium (go above for margin).
- Risk Factors: What could invalidate this recommendation? (seasonal demand, low review counts, product being discontinued.)
- Confidence Level: How reliable is this recommendation given the data completeness?

4. RECOMMENDED TECH STACK

Layer	Technology / Tool	Purpose
Vision AI	GPT-4o Vision / Claude 3.5 Sonnet / LLaVA (Ollama)	Image-to-product identification
Web Scraping	Playwright (async) + httpx + BeautifulSoup4	Headless browser + lightweight HTTP scrapers
Stealth	playwright-stealth + fake-useragent	Anti-bot evasion across JS-heavy sites
Task Queue	Celery + Redis OR asyncio.TaskGroup	Parallel scraper orchestration
Data Store	TimescaleDB (PostgreSQL extension) + Redis	Price history time-series + session hot cache
ML Engine	scikit-learn (KMeans, IsolationForest) + pandas	Price clustering and anomaly detection
GenAI Layer	OpenAI GPT-4o / Anthropic Claude / Ollama	Streaming pricing rationale generation

Backend API	FastAPI + Uvicorn + sse-starlette	Async API gateway with SSE streaming support
Frontend	React + Vite + TailwindCSS + Recharts	Real-time streaming dashboard + charts
Containerisation	Docker + Docker Compose	Single-command local deployment of all services
Observability	Prometheus + Grafana + Loki + Promtail	Metrics, dashboards, structured logs
Image Handling	Pillow + python-magic	Image validation, format detection, preprocessing

5. DATA COLLECTION & STREAMING STRATEGY

5.1 On-Demand Data Collection Philosophy

Unlike Project 1 (which uses a background data pipeline), this system collects ALL data on-demand at query time. There is no background ingestion process. Every user query triggers a fresh round of live scraping. This means:

- Data is always fresh — prices are guaranteed to be from the current session.
- The system must be resilient to partial failures — if 2 of 6 scrapers fail, produce the best possible recommendation from the remaining 4.
- The system must communicate partial results clearly to the user — show which sites succeeded, which failed, and why.

5.2 Streaming Event Protocol

Every scraper publishes events to a Redis channel or Celery result backend. The FastAPI SSE endpoint subscribes and forwards these events to the connected browser client in real time:

```
// ── Event Type 1: price_scraped (success) ──────────────────────
{
  "event"            : "price_scraped",
  "session_id"       : "a1b2c3d4-uuid",
  "source"           : "amazon",
  "product_name"     : "Apple AirPods Pro 2nd Gen USB-C",
  "price"            : 189.99,
  "currency"         : "USD",
  "seller_rating"    : 4.7,
  "review_count"     : 48201,
  "in_stock"         : true,
  "product_url"      : "https://amazon.com/dp/XXXXXXXXXX",
  "scraped_at"       : "2025-09-15T10:23:45Z",
  "latency_ms"       : 2340
}

// ── Event Type 2: scraper_failed ────────────────────────────────
{
```

```
  "event"            : "scraper_failed",
  "session_id"       : "a1b2c3d4-uuid",
  "source"           : "walmart",
  "reason"           : "CAPTCHA_DETECTED",
  "retry_count"      : 3,
  "failed_at"        : "2025-09-15T10:23:52Z"
}

// ── Event Type 3: analysis_ready ──────────────────────────────
{
  "event"            : "analysis_ready",
  "session_id"       : "a1b2c3d4-uuid",
  "recommended_price": 179.99,
  "price_range"      : { "low": 159.99, "high": 199.99 },
  "strategy"         : "competitive",
  "competitive_score": 78
}
```

5.3 Price History & Trend Storage

TimescaleDB schema must support the following query patterns:

- 'Show all prices scraped for this product over the last 30 days' — requires time-series indexing on product_name + scraped_at.
- 'Which site has historically offered the lowest price?' — requires aggregation by source.
- 'Is the price trending up or down?' — requires computing price velocity (slope of price over time).

6. USER INTERFACE REQUIREMENTS

6.1 Application Screens

Screen 1 — Product Upload & Vision AI Identification

- Large drag-and-drop upload zone with image preview and file format label (JPG, PNG, WebP, HEIC).
- Upon upload: show 'Analysing image with Vision AI...' skeleton loader.
- Once Vision AI responds, render a Product Identity Card:
  - Product name + brand (large, prominent).
  - Category hierarchy (e.g., Electronics > Audio > Wireless Earbuds).
  - Key specs extracted from the image as tags/chips.
  - Confidence score badge — green if >= 0.80, yellow if 0.60–0.79, red if < 0.60.
  - 'Edit Details' button for manual correction if AI got it wrong.
- 'Find Best Price Now' CTA button — triggers the scraping pipeline.

Screen 2 — Live Scraping Dashboard (Core Screen — Most Important)

- Scraper Status Row: A horizontal row of site logos, each with a live status chip — Queued (grey) → Scraping (animated green spinner) → Found (green tick + price) → Failed (red X + reason).
- Price Cards Grid: Animates in one-by-one as each scraper returns. Each card shows: site logo, scraped price (large), seller rating, review count, stock status, and a 'View Listing' link.
- Live Price Distribution Chart (bar/histogram): Updates with each new data point. The chart must rebuild in real time as new prices arrive — not render all at once at the end.

- Market Stats Bar: Dynamically updates — shows current lowest price, average price, highest price, and count of sources found.
- Product Validation Indicator: Shows whether each scraped product title was verified to match the uploaded product (green = match confirmed, yellow = uncertain).

Screen 3 — Pricing Recommendation Panel

- Renders once at least 3 sources have returned prices (configurable threshold).
- Recommended Price Badge: Large, prominent, colour-coded — green (competitive), gold (premium).
- Price Range Visualiser: A horizontal bar showing where the recommended price sits between the market minimum and maximum.
- Pricing Strategy Label: 'Penetration Pricing' / 'Competitive Pricing' / 'Premium Pricing'.
- Streaming GenAI Rationale Panel: Text appears token-by-token with a blinking cursor. The five sections (Market Summary, Recommended Price, Strategy, Risk Factors, Confidence) should be visually separated.
- Competitive Score Meter: A 0–100 gauge showing how well-positioned the price is vs. competitors.

Screen 4 — Price History & Trends

- Time-series line chart: All historical prices for this product across all past queries.
- Per-source trend lines in different colours — which site has been cheapest over time?
- Price Velocity Indicator: Is the price trending up, down, or stable? (calculated from slope of the trend line.)
- If no history exists (first query for this product), display a 'No history yet — this is the first time this product was queried' message.

Screen 5 — Observability Dashboard

- Embedded Grafana panels or custom chart components showing: scraper success rates by site, vision AI latency, ML analysis latency, GenAI time-to-first-token, total end-to-end query latency.
- Circuit breaker status table: which sites are currently in 'open' (blocked) state and when they will reset.

6.2 UX Requirements & Performance Targets

- Vision AI result (product identity card) must appear within 6 seconds of image upload.
- First price card must appear within 10 seconds of clicking 'Find Best Price'.
- All scrapers must either complete or time out within 60 seconds maximum.
- The GenAI recommendation must begin streaming within 3 seconds of the ML analysis completing.
- All scraper failure states must be shown clearly in the UI — no silent failures.
- The UI must be fully responsive for desktop and tablet viewports.

7. OBSERVABILITY & METRICS

Metric	Description	Tool	Alert Threshold

vision_ai_latency_ms	Time for Vision LLM to identify product from uploaded image	Prometheus Histogram	> 8s = alert
vision_ai_confidence_score	Distribution of confidence scores across all product queries	Prometheus Histogram	mean < 0.70 = alert
scraper_latency_ms	Per-site scraping duration (label: site_name)	Prometheus Histogram	> 25s = timeout
scraper_success_rate	Percentage of scrape attempts returning a valid, verified price	Prometheus Gauge	< 50% = alert
scraper_failures_total	Count of scraper failures by reason and site name	Prometheus Counter	Informational
price_anomaly_rate	Percentage of scraped prices flagged as anomalous by ML	Prometheus Gauge	> 30% = alert
ml_analysis_latency_ms	Time for ML engine to complete price clustering and analysis	Prometheus Histogram	> 2s = alert
genai_time_to_first_token_ms	Latency from ML ready to first streaming token in the UI	Prometheus Gauge	> 3s = alert
end_to_end_query_latency_ms	Total: image upload to first price card displayed in UI	Prometheus Histogram	> 15s = alert
price_history_records_total	Total price data points stored in TimescaleDB	Prometheus Counter	Informational

7.2 Grafana Dashboards

- Dashboard 1 — Scraper Health: Success rate per site (bar chart), failure breakdown by reason (pie chart), scraper latency p50/p90/p99, circuit breaker status per site, scraper uptime over 24 hours.
- Dashboard 2 — Vision AI & ML Performance: Confidence score distribution histogram, vision latency over time, ML analysis latency, price anomaly rate trend, K-Means cluster distribution.
- Dashboard 3 — End-to-End System: Total query latency histogram, GenAI time-to-first-token, concurrent active sessions, prices scraped per hour, TimescaleDB write throughput.

7.3 Logging Requirements

- All services must emit structured JSON logs with fields: timestamp, service, session_id, level, event, metadata.
- session_id must propagate through all stages from image upload to final GenAI token.
- Scraper logs must include: site, search_query_used, http_status_code, price_found, product_title_match_score, latency_ms.

- Vision AI logs must include: model_used, image_size_kb, product_identified, confidence_score, cache_hit (true/false).
- All logs must be shipped to Loki via Promtail and queryable via LogQL in Grafana.

8. 2-WEEK PROJECT TIMELINE

Week	Days	Milestone	Deliverable
Week 1	Day 1	Environment & Skeleton Setup	Docker Compose: FastAPI, Redis, TimescaleDB, Prometheus, Grafana all running
Week 1	Day 1–2	Vision AI Integration	Image upload endpoint → Vision LLM → structured JSON product schema working
Week 1	Day 2–3	First 2 Scrapers (Amazon + eBay)	Both scrapers returning normalised prices, stored in TimescaleDB
Week 1	Day 3–4	3 More Scrapers + Orchestrator	5+ scrapers running in parallel; SSE events published on each completion
Week 1	Day 4–5	Anti-bot Hardening	Rotating agents, stealth, circuit breakers, retry + jitter logic working
Week 1	Day 5–6	ML Price Analysis Engine	KMeans + IsolationForest microservice live; outputs recommendation object
Week 1	Day 6–7	GenAI Streaming Recommendation	FastAPI SSE → LLM → tokens visible in Postman / WebSocket test client
Week 2	Day 8	React UI — Upload + Vision Card	Image upload, Vision AI product card, confidence badge all rendered
Week 2	Day 9	React UI — Live Scraping Dashboard	Price cards stream in real time; scraper status row animates per site
Week 2	Day 10	React UI — Recommendation + History	Streaming GenAI panel, price range bar, price history trend chart working
Week 2	Day 11	Observability Stack Complete	All 10 metrics live; 3 Grafana dashboards with alert rules configured
Week 2	Day 12	Integration & Performance Testing	pytest + Playwright E2E tests passing; latency targets verified
Week 2	Day 13	Documentation & Demo Prep	README, architecture diagram, demo image set, slide deck finalised

Week 2	Day 14	Final Demo Day	Live demo with full pipeline, Grafana walkthrough, and Q&A

9. DEMO REQUIREMENTS & EVALUATION CRITERIA

9.1 Demo Structure (Day 14 — 50 Minutes)

Part 1 — Architecture Walkthrough (10 minutes)

Present your architecture diagram. Be ready to answer:

- How did you choose which Vision LLM to use? Did you benchmark multiple options?
- What is your scraper orchestration strategy — how does the system decide which scrapers to launch?
- What happens if 4 out of 6 scrapers fail? Walk through the failure path end-to-end.
- Why did you choose your specific anti-bot techniques? What did you try that did not work?
- How does the ML engine compute the recommended price — walk through the exact calculation.

Part 2 — Live Demo (25 minutes)

All demos use real images uploaded live. Pre-loaded images are allowed, but scraping must be live and real.

1. Upload a laptop image — show Vision AI identifying brand, model, generation, and key specs.
2. Click 'Find Best Price' — show all scraper status indicators animate from Queued to Scraping.
3. Watch price cards appear one-by-one as each scraper returns — demonstrate true streaming.
4. Show the price distribution chart building in real time with each new data point.
5. Show the ML analysis output: clusters, outliers flagged, recommended price range.
6. Watch the GenAI recommendation stream token-by-token in the rationale panel.
7. Upload a second product image (e.g., a smartwatch) — show the system handling a different category.
8. Upload a blurry / ambiguous image — show the system's low-confidence warning and graceful handling.
9. Show the Grafana dashboards: scraper success rates, vision latency, end-to-end latency.

Part 3 — Methodology Deep Dive (10 minutes)

- Show the Vision LLM prompt you designed — explain every line and why it produces consistent JSON output.
- Walk through one scraper's code — show anti-bot evasion at the code level (user-agent rotation, jitter, stealth).
- Explain how product title validation works — how do you verify the scraped product is actually the correct one?
- Show the TimescaleDB schema — explain the indexing strategy for price history queries.
- Walk through the ML clustering output for a real query — show the cluster centroids and which tier the recommended price is in.

Part 4 — Q&A (5 minutes)

Open questions on any aspect of the system including code, design decisions, and failure modes.

9.2 Scoring Rubric

Criterion	What We Look For	Max Score
Vision AI — Product Identification	Accuracy across product types, low-confidence handling, caching, JSON schema compliance	20
Web Scraping — Coverage & Robustness	5+ sites, anti-bot working, streaming events per site, circuit breakers, product validation	25
ML Price Analysis	Clustering correct, outlier detection working, valid recommendation + confidence interval	20
GenAI Streaming Recommendation	Quality of rationale, streaming visible in UI, 5 required sections present, prompt design	15
UI — Real-Time Streaming Experience	Cards stream live, all states handled (loading/failed/empty), chart updates incrementally	10
Observability	All 10 metrics live, 3 dashboards, structured logs with session_id propagated end-to-end	10
TOTAL | Minimum passing score: 70 / 100		**100**

10. DELIVERABLES CHECKLIST

10.1 Code Repository

```
dynamic-pricing-platform/
  ├── vision-service/           # Vision AI product identification
  │     ├── app.py              # FastAPI endpoint + LLM vision client
  │     ├── prompt.py           # Vision prompt engineering + output parser
  │     ├── cache.py            # MD5-based image result cache (Redis)
  │     └── Dockerfile
  ├── scraping-service/
  │     ├── orchestrator.py     # Async task launcher + SSE event publisher
  │     ├── scrapers/           # One file per site: amazon.py, ebay.py, ...
  │     ├── normaliser.py       # Price normalisation + anomaly flagging
  │     ├── circuit_breaker.py  # Per-site circuit breaker logic
  │     └── Dockerfile
  ├── ml-service/
  │     ├── analyser.py         # KMeans clustering + IsolationForest
  │     ├── demand.py           # Demand signal estimation from review data
  │     └── Dockerfile
  ├── genai-service/
  │     ├── recommender.py      # Prompt builder + LLM streaming client
  │     └── Dockerfile
  ├── api-gateway/
```

```
│     ├── main.py             # FastAPI routes + SSE orchestration
│     ├── metrics.py          # Prometheus metrics definitions
│     └── Dockerfile
├── frontend/
│     ├── src/
│     │     ├── components/   # UploadZone, PriceCard, ScraperStatus, ...
│     │     └── pages/        # Dashboard, History, Observability
│     └── Dockerfile
├── observability/
│     ├── prometheus.yml
│     └── grafana/dashboards/ # JSON dashboard exports
├── tests/                    # pytest unit + integration + Playwright E2E
├── docker-compose.yml
├── .env.example
└── README.md
```

10.2 Documentation

- ☐ README.md: Setup guide, architecture overview, environment variables, how to run tests.
- ☐ Architecture diagram (Excalidraw / draw.io / Miro) exported as PNG and committed to repo.
- ☐ Scraper Report: For each site — observed success rate during testing, anti-bot challenges encountered, techniques used, known failure modes.
- ☐ Vision AI Evaluation: Test results on 20 diverse product images — accuracy, failure cases, confidence distribution.

10.3 Demo Materials

- ☐ A slide deck (max 8 slides) for the architecture walkthrough portion.
- ☐ A curated test image set of 6 products: electronics, clothing, appliance, book, footwear, and one deliberately low-quality/ambiguous image.

11. TIPS, HINTS & COMMON PITFALLS

11.1 Tips for Success

- ☐ **Test your Vision AI prompt on 10 different product images before building anything else. It is the entire foundation. If identification is unreliable, everything downstream will be unreliable.**
- ☐ **Build and test scrapers one at a time. Get Amazon working first, then eBay, then tackle the harder sites. Do not try to build all scrapers on the same day.**
- ☐ **Use Playwright's network interception (page.route + page.on('response')) to capture XHR responses — this is often faster and more stable than parsing HTML.**
- ☐ **Design your SSE event schema on Day 1 and do not change it. Changing the schema later will simultaneously break the backend publisher and the React SSE consumer.**
- ☐ **Implement the circuit breaker early. Without it, a single unavailable site will cause every query to wait for its full 25-second timeout, making the system feel very slow.**
- ☐ **Cache Vision AI results by image MD5 hash from Day 2. The Vision LLM is the most expensive call in the pipeline — never call it twice for the same image.**

11.2 Common Pitfalls to Avoid

- Scraping only 2–3 sites — you need at least 5 for meaningful price clustering. With fewer than 3 data points, KMeans produces meaningless clusters.
- Not validating that the scraped product matches the uploaded product — Amazon search for 'AirPods Pro' can return cases, chargers, and accessories, not just the earbuds.
- Running Playwright synchronously inside FastAPI — this blocks the event loop. Always use asyncio + Playwright async API or run in a thread pool executor.
- Using the same user-agent string for parallel scrapers — multiple simultaneous requests with the same UA string is a bot detection signature.
- Not handling empty price history gracefully — the trend chart will crash if there is no historical data for a product. Always handle empty state explicitly.
- Showing a recording or mock data in the demo — the assessor will upload a product image they choose during the demo. The system must handle unknown products live.

11.3 Learning Resources

- Playwright Python async documentation: playwright.dev/python/docs/api/class-page
- GPT-4o Vision API guide: platform.openai.com/docs/guides/vision
- sse-starlette for FastAPI SSE: github.com/sysid/sse-starlette
- TimescaleDB getting started: docs.timescale.com/getting-started
- scikit-learn KMeans + IsolationForest: scikit-learn.org/stable/modules/clustering.html
- fake-useragent Python library: github.com/fake-useragent/fake-useragent

A NOTE FROM YOUR ASSESSOR

The hardest part of this project is not the ML model or the GenAI integration.
It is making scrapers that actually work in the real world —
where websites change layouts, enforce CAPTCHAs, and block bots without warning.
We do not expect every scraper to work perfectly. We expect resilience:
graceful failures, clear observability into what broke and why,
and a system that produces a useful answer even when some sources are unavailable.

Find the best price. Build something sharp.